

CS222

Operating Systems

Lecture 13

I/O Devices, HDD, SSD, RAID

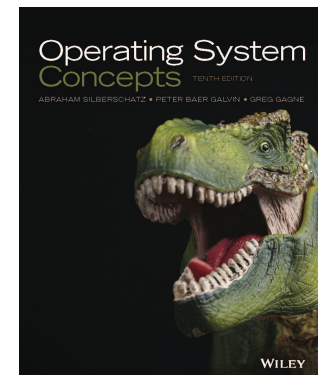
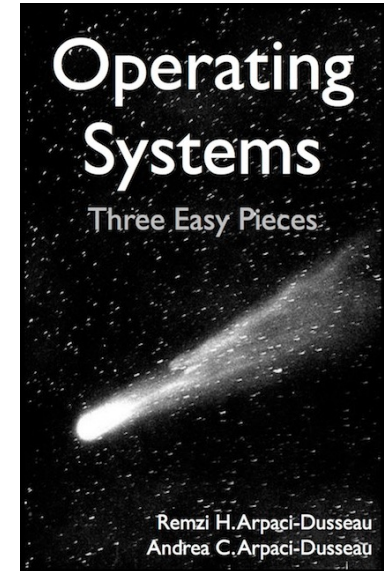
(Section 100001)

ผศ. ดร. กษิธิศ ชาญเขียว

ckasidit@tu.ac.th

Textbooks

- Operating Systems: Three Easy Pieces
 - Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau
 - Arpaci-Dusseau Books, November, 2023 (Version 1.10)
- Avi Silberschatz, Peter B. Galvin and Greg Gagne; Operating System Concepts, 10th Edition; John Wiley & Sons, Inc;
- Original Slides
- <https://www.os-book.com/OS9/slide-dir/index.html>
- <https://www.os-book.com/OS10/slide-dir/index.html>



Prototypical System Architecture

- สถาปัตยกรรมแบบแบบทั่วไป
- อุปกรณ์ความเร็วสูงต่อกับ PCI bus
- อุปกรณ์ความเร็วต่ำต่อกับ Peripheral I/O bus

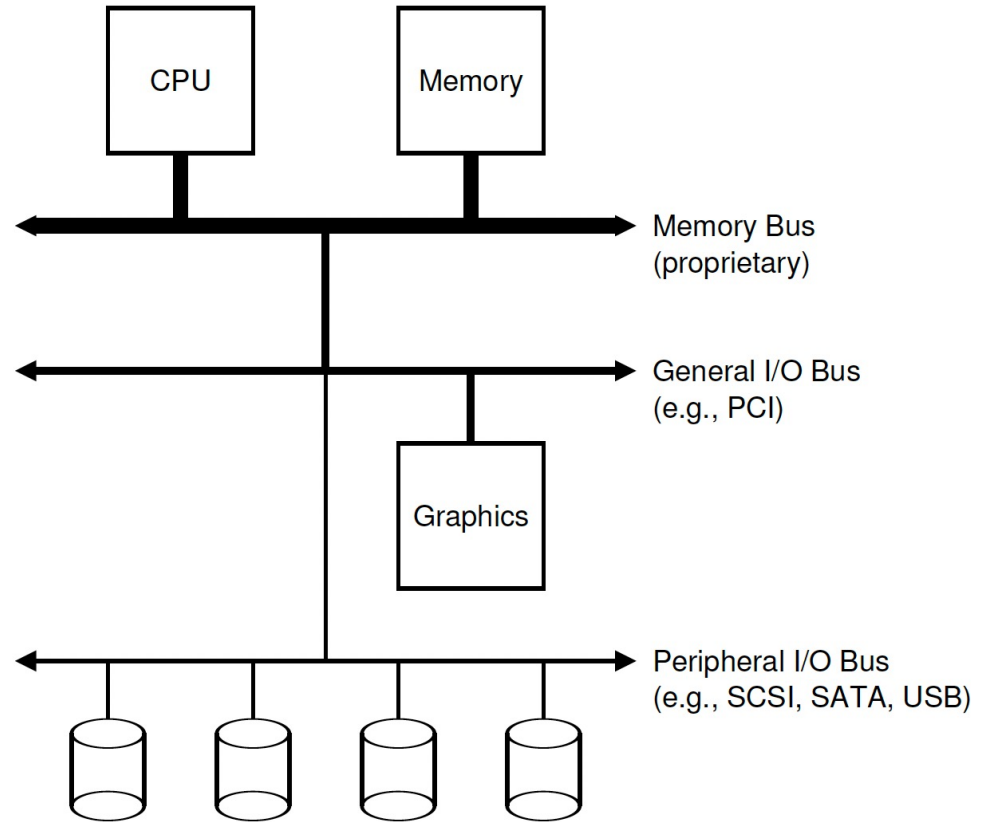


Figure 36.1: Prototypical System Architecture



Modern System Architecture

- สถาปัตยกรรมแบบใหม่ เชื่อม CPU กับ Mem และเชื่อม CPU เข้ากับ GPU (เอาใจ Gamer) เพื่อการแสดงผลที่เร็วและประมวลผลขั้นสูง
- Intel มี I/O Chip ดูแลอุปกรณ์
- CPU ส่งข้อมูลและคำสั่งผ่าน Direct Media Interface ไปยัง I/O Chip

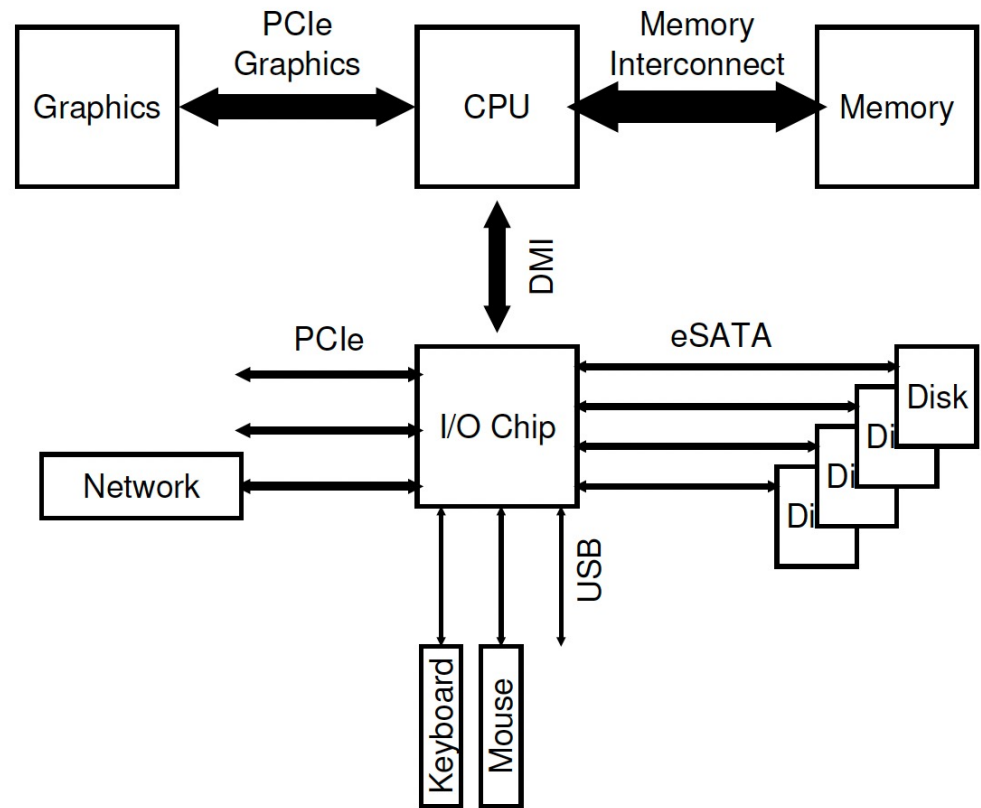


Figure 36.2: Modern System Architecture





Modern System Architecture

- จัดการ HDD ผ่าน eSATA interface
วิวัฒนาการ
 - AT Attachment (ATA)
 - Serial ATA (SATA)
 - external SATA (eSATA)
- จัดการ Mouse, Keyboard, อุปกรณ์ USB (ผ่าน USB interface)
- I/O Chip สามารถเชื่อมต่ออุปกรณ์ความเร็วสูงเช่น Network Interface Card (NIC) หรือ NVMe drive (SSD) ผ่าน PCIe Interface ได้ด้วย

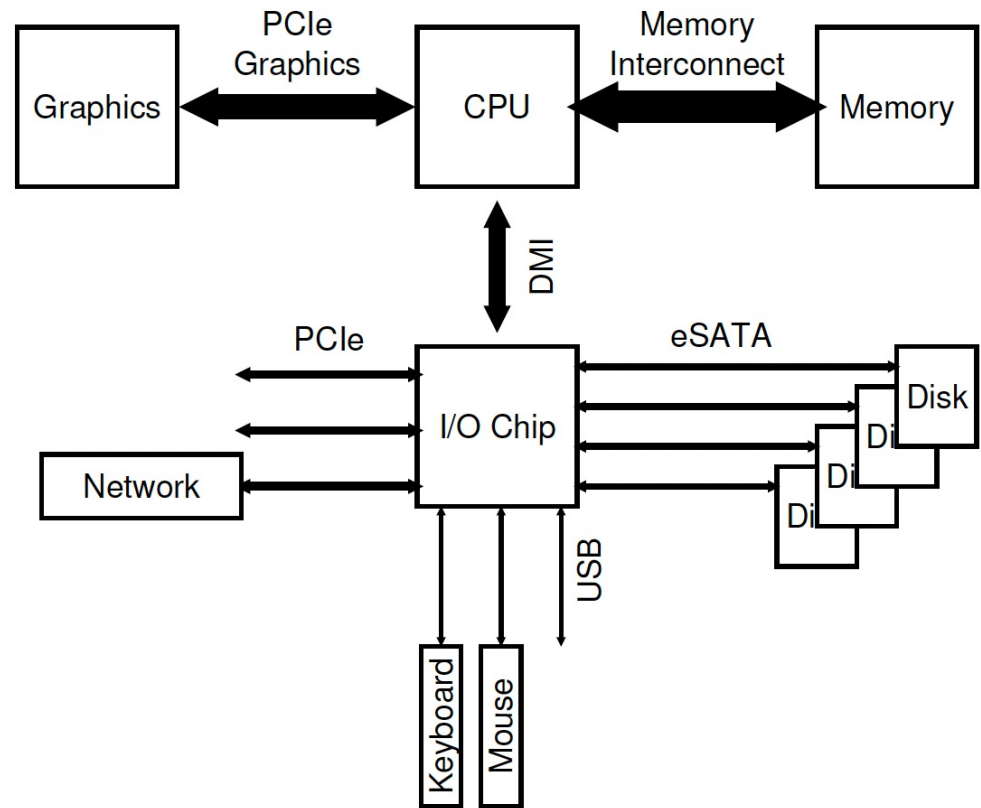


Figure 36.2: Modern System Architecture





Canonical Device

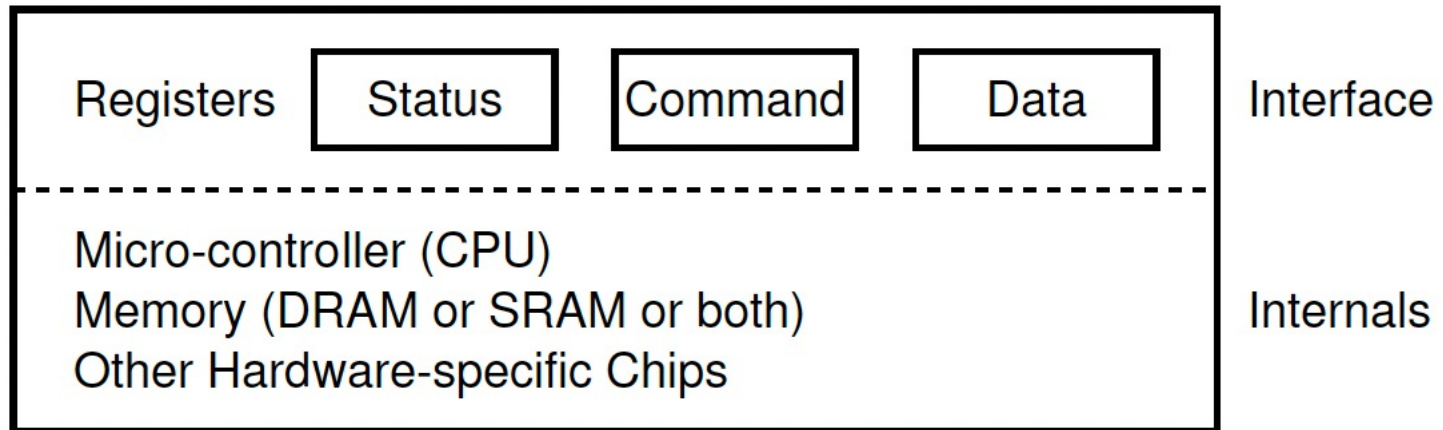


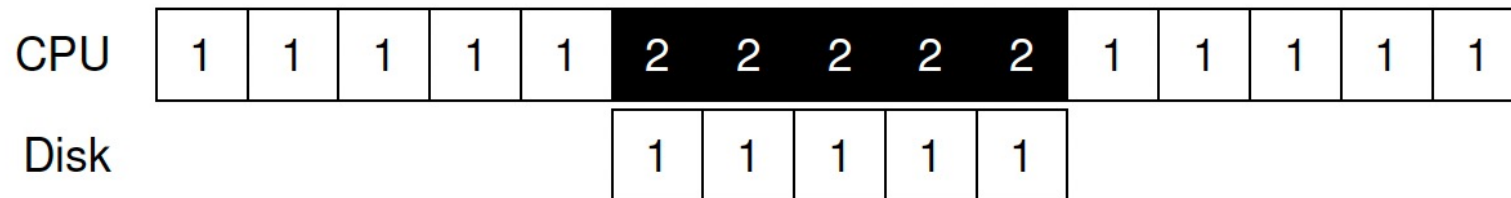
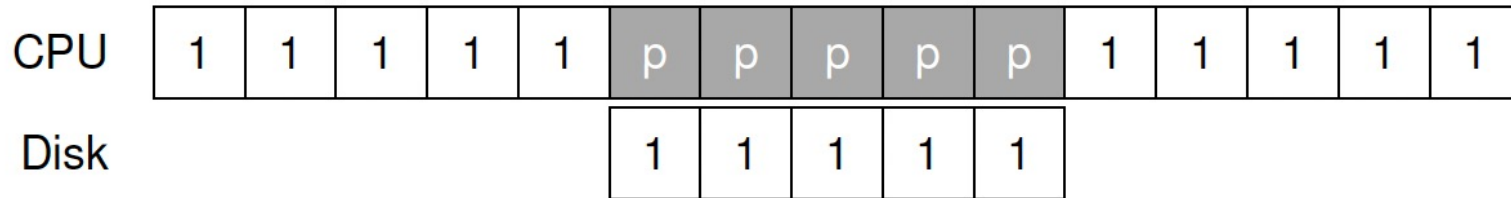
Figure 36.3: A Canonical Device

- โครงสร้างของอุปกรณ์ I/O ประกอบไปด้วย Interface และ Internals
- Interfaces ประกอบไปด้วย registers สำหรับเก็บค่า status รับ command และรับ data
- Internals เป็นตั้งแต่วงจรเล็กๆ ถึงคอมพิวเตอร์ขนาดเล็กที่มี CPU ของตนเอง





CPU – I/O Interactions

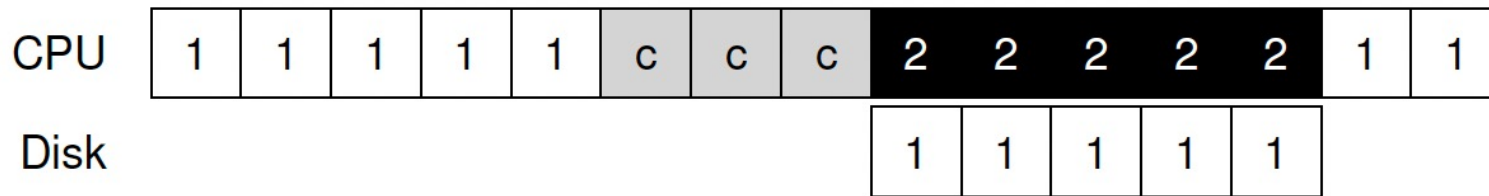


- Programmed I/O: หลังจากส่งงานไปให้ I/O ซีพียู poll เช็คค่า STATUS เป็นระยะๆ ว่า I/O เสร็จหรือยัง ถ้าเสร็จก็จะให้ Process ทำงานต่อได้ CPU ไม่ได้ใช้งานระหว่างรอ I/O
- I/O Interrupts: เพิ่ม utilization และ

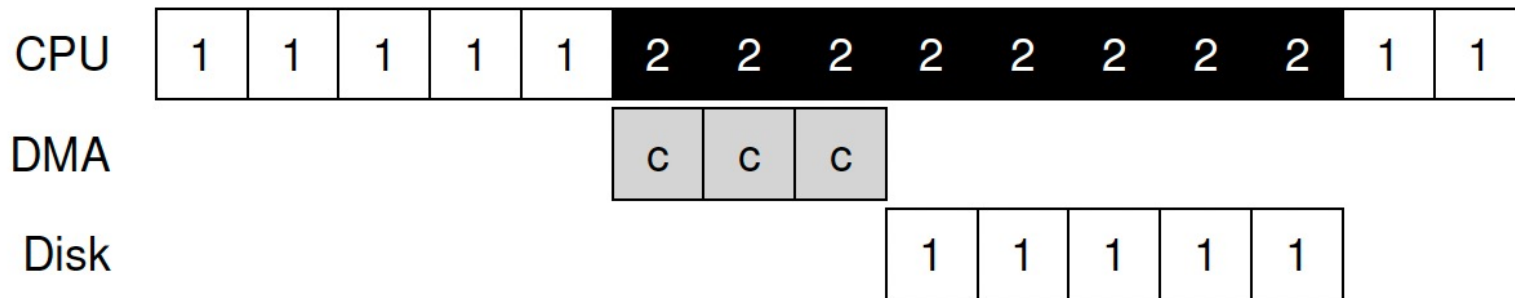




CPU – I/O Interaction: DMA helps



■ แบบไม่มี DMA



■ DMA ช่วย transfer ข้อมูลระหว่าง memory กับ I/O devices





Device Driver

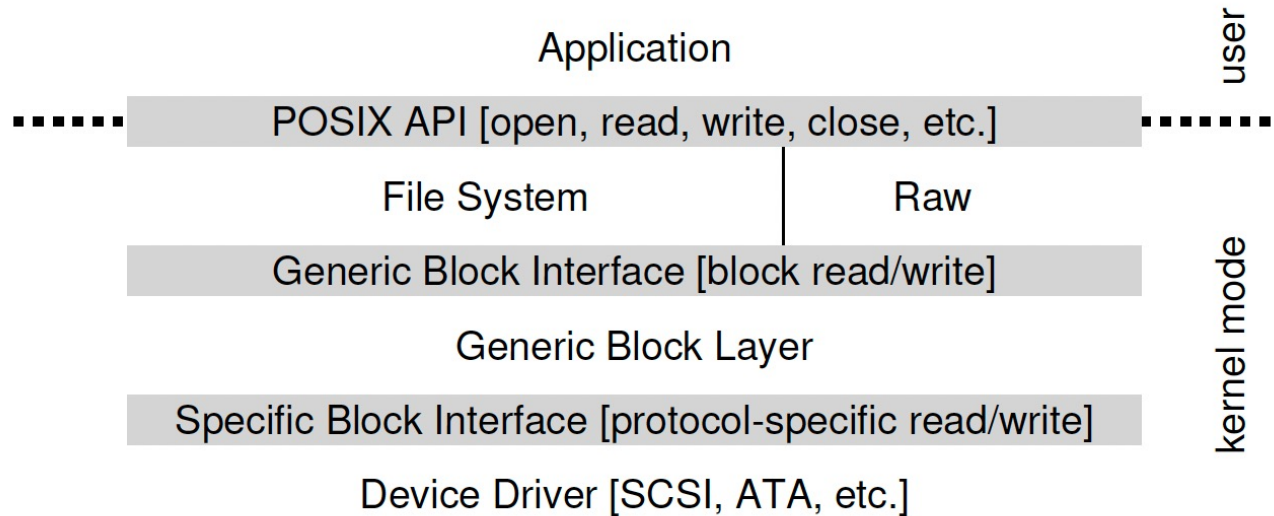


Figure 36.4: The File System Stack

- Device Driver เป็น Abstract Layer ใน File System Stack ของ OS
- ทำหน้าที่บรรจุ routine สำหรับจัดการอุปกรณ์
- Generic Layer อาจไม่ครอบคลุม function ทั้งหมดของอุปกรณ์ แต่ทำให้การเปลี่ยน Device และ Device Driver ไม่กระทบโครงสร้าง OS





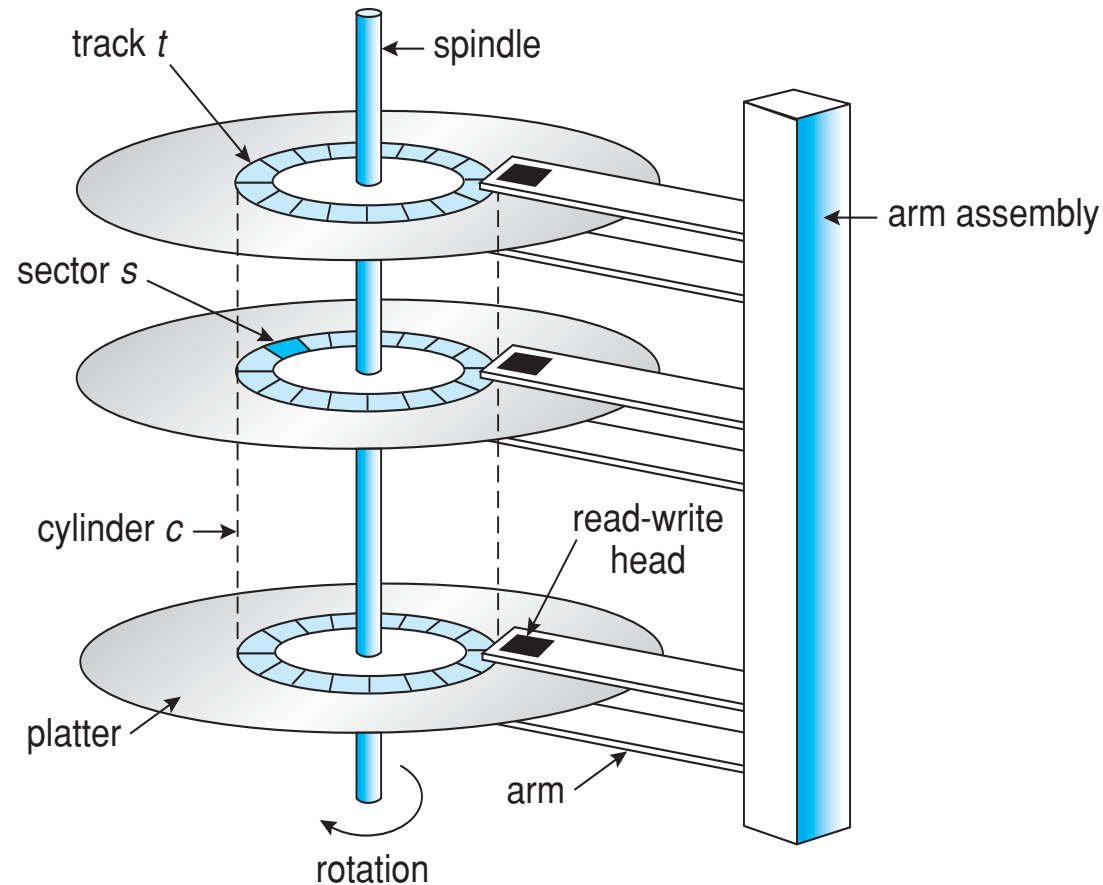
Overview of Mass Storage Structure

- Bulk of secondary storage for modern computers is **hard disk drives (HDDs)** and **nonvolatile memory (NVM)** devices
- **HDDs** spin platters of magnetically-coated material under moving read-write heads
 - Drives rotate at 60 to 250 times per second
 - **Transfer rate** is rate at which data flow between drive and computer
 - **Positioning time (random-access time)** is time to move disk arm to desired cylinder (**seek time**) and time for desired sector to rotate under the disk head (**rotational latency**)
 - **Head crash** results from disk head making contact with the disk surface -- That's bad
- Disks can be removable





Moving-head Disk Mechanism





Hard Disk Drives

- Platters range from .85" to 14" (historically)
 - Commonly 3.5", 2.5", and 1.8"
- Sector size เดิม 512 ปัจจุบัน 4096 bytes
- Performance
 - Transfer Rate – theoretical – 6 Gb/sec
 - Effective Transfer Rate – real – 1Gb/sec
 - Seek time from 3ms to 12ms – 9ms common for desktop drives
 - Average seek time measured or calculated based on 1/3 of tracks
 - Latency based on spindle speed
 - ▶ $1 / (\text{RPM} / 60) = 60 / \text{RPM}$
 - Average latency = $\frac{1}{2}$ latency



- Common drives spin at 5,400, 7,200, 10,000, and 15,000 RPM





Hard Disk Performance

- **Seek time** = time to move read-write head over desired cylinder
- **Rotation latency** = time to get to the desired sector
- **Access Latency** = **Average access time** = average seek time + average latency
 - For fastest disk $3\text{ms} + 2\text{ms} = 5\text{ms}$
 - For slow disk $9\text{ms} + 5.56\text{ms} = 14.56\text{ms}$
- Average I/O time = average access time + (amount to transfer / transfer rate) + controller overhead
- For example to transfer a 4KB block on a 7200 RPM disk with a 5ms average seek time, 1Gb/sec transfer rate with a .1ms controller overhead =
 - $5\text{ms} + 4.17\text{ms} + 0.1\text{ms} + \text{transfer time} =$
 - Transfer time = $4\text{KB} / 1\text{Gb/s} * 8\text{Gb} / \text{GB} * 1\text{GB} / 1024^2\text{KB} = 32 / (1024^2) = 0.031 \text{ ms}$
 - Average I/O time for 4KB block = $9.27\text{ms} + .031\text{ms} = 9.301\text{ms}$





The First Commercial Disk Drive



1956
IBM RAMDAC computer
included the IBM Model 350
disk storage system

5M (7 bit) characters
50 x 24" platters
Access time = < 1 second





Nonvolatile Memory Devices

- If disk-drive like, then called **solid-state disks (SSDs)**
- Other forms include **USB drives** (thumb drive, flash drive), DRAM disk replacements, surface-mounted on motherboards, and main storage in devices like smartphones
- Can be more reliable than HDDs
- More expensive per MB
- Maybe have shorter life span – need careful management
- Less capacity
- But much faster
- Busses can be too slow -> connect directly to PCI for example
- No moving parts, so no seek time or rotational latency





Nonvolatile Memory Devices

- Have characteristics that present challenges
- Read and written in “page” increments (think sector) but can’t overwrite in place
 - Must first be erased, and erases happen in larger “block” increments
 - Can only be erased a limited number of times before worn out – ~ 100,000
 - Life span measured in **drive writes per day (DWPD)**
 - ▶ A 1TB NAND drive with rating of 5DWPD is expected to have 5TB per day written within warrantee period without failing





NAND Flash Controller Algorithms

- With no overwrite, pages end up with mix of valid and invalid data
- To track which logical blocks are valid, controller maintains **flash translation layer (FTL)** table
- Also implements **garbage collection** to free invalid page space
- Allocates **overprovisioning** to provide working space for GC
- Each cell has lifespan, so **wear leveling** needed to write equally to all cells

valid page	valid page	invalid page	invalid page
invalid page	valid page	invalid page	valid page

NAND block with valid and invalid pages





Writes to NAND Storage

FTL	
logical address	physical address
1	
2	
3	
4	
5	
6	
7	
8	

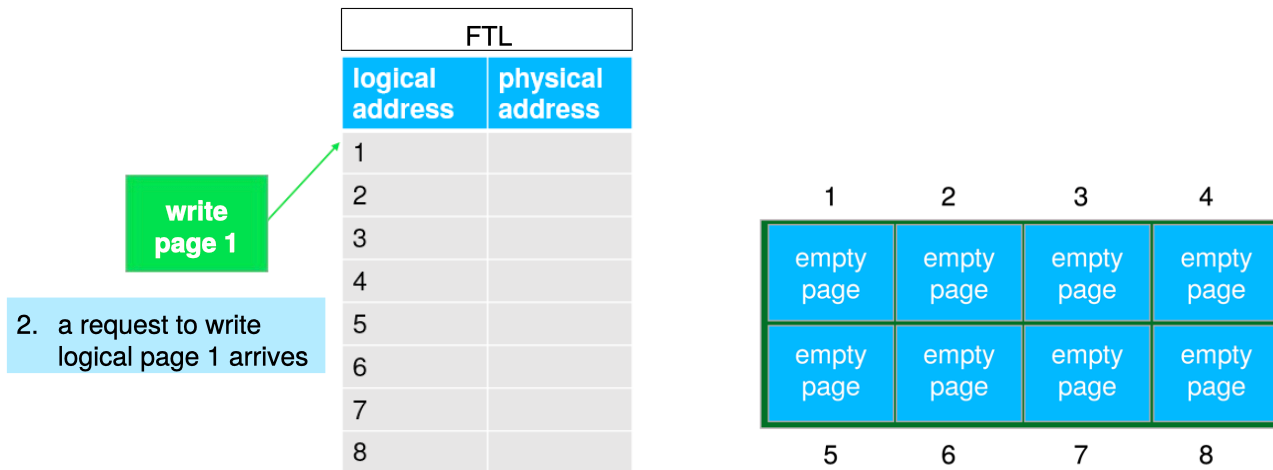
1	2	3	4
empty page	empty page	empty page	empty page
empty page	empty page	empty page	empty page
5	6	7	8

1. assume eight pages in one block of NAND flash storage, not yet written to



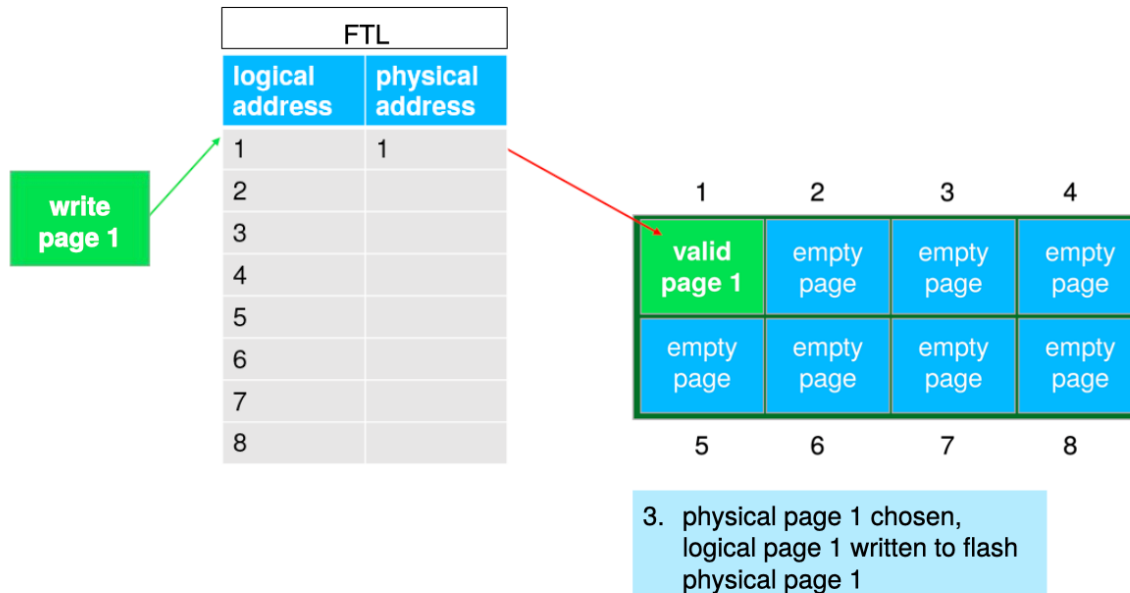


Writes to NAND Storage



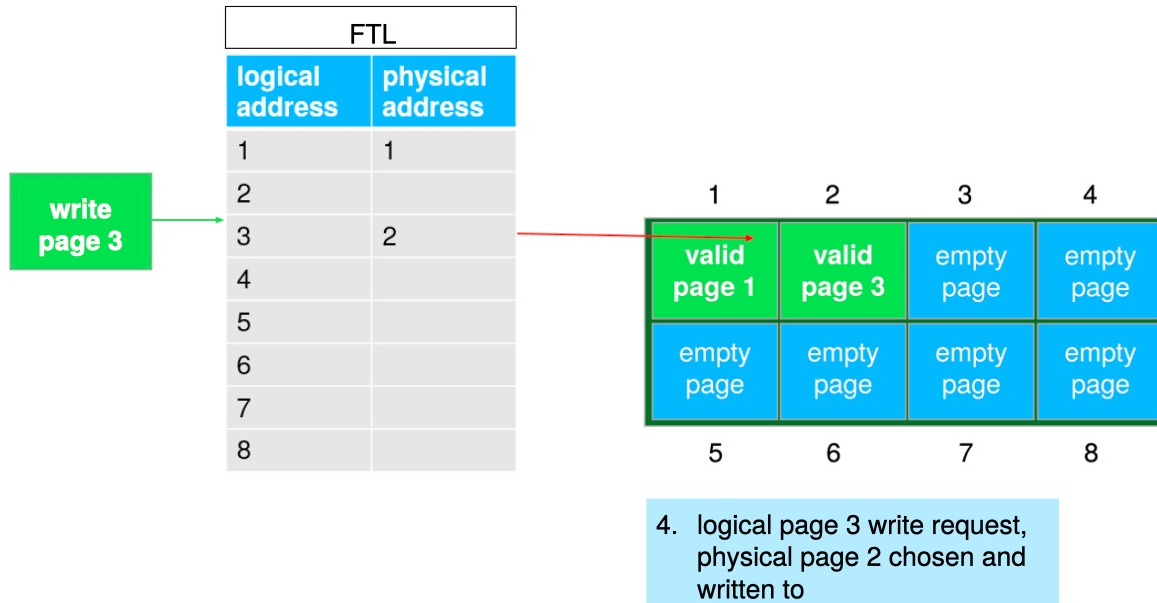


Writes to NAND Storage



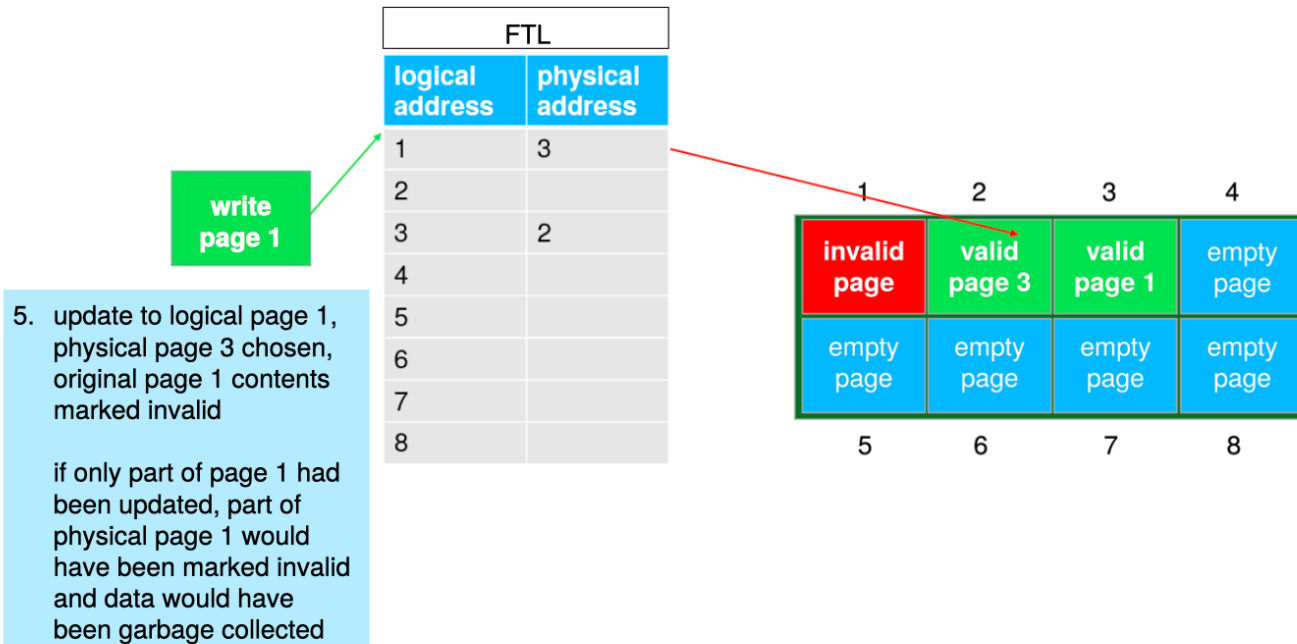


Writes to NAND Storage





Writes to NAND Storage





Volatile Memory

- DRAM frequently used as mass-storage device
 - Not technically secondary storage because volatile, but can have file systems, be used like very fast secondary storage
- **RAM drives** (with many names, including RAM disks) present as raw block devices, commonly file system formatted
- Computers have buffering, caching via RAM, so why RAM drives?
 - Caches / buffers allocated / managed by programmer, operating system, hardware
 - RAM drives under user control
 - Found in all major operating systems
 - ▶ Linux `/dev/ram`, macOS `diskutil` to create them, Linux `/tmp` of file system type `tmpfs`
- Used as high speed temporary storage
 - Programs could share bulk data, quickly, by reading/writing to RAM drive





Magnetic Tape

Magnetic tape was used as an early secondary-storage medium. Although it is nonvolatile and can hold large quantities of data, its access time is slow compared with that of main memory and drives. In addition, random access to magnetic tape is about a thousand times slower than random access to HDDs and about a hundred thousand times slower than random access to SSDs so tapes are not very useful for secondary storage. Tapes are used mainly for backup, for storage of infrequently used information, and as a medium for transferring information from one system to another.

A tape is kept in a spool and is wound or rewound past a read–write head. Moving to the correct spot on a tape can take minutes, but once positioned, tape drives can read and write data at speeds comparable to HDDs. Tape capacities vary greatly, depending on the particular kind of tape drive, with current capacities exceeding several terabytes. Some tapes have built-in compression that can more than double the effective storage. Tapes and their drivers are usually categorized by width, including 4, 8, and 19 millimeters and 1/4 and 1/2 inch. Some are named according to technology, such as LTO-6 (Figure 11.5) and SDLT.



Figure 11.5 An LTO-6 Tape drive with tape cartridge inserted.





Disk Attachment

- Host-attached storage accessed through I/O ports talking to **I/O busses**
- Several busses available, including **advanced technology attachment (ATA)**, **serial ATA (SATA)**, **eSATA**, **serial attached SCSI (SAS)**, **universal serial bus (USB)**, and **fibre channel (FC)**.
- Most common is SATA
- Because NVM much faster than HDD, new fast interface for NVM called **NVM express (NVMe)**, connecting directly to PCI bus
- Data transfers on a bus carried out by special electronic processors called **controllers** (or **host-bus adapters, HBAs**)
 - Host controller on the computer end of the bus, device controller on device end
 - Computer places command on host controller, using memory-mapped I/O ports
 - ▶ Host controller sends messages to device controller
 - ▶ Data transferred via DMA between device and computer DRAM





Address Mapping

- Disk drives are addressed as large 1-dimensional arrays of **logical blocks**, where the logical block is the smallest unit of transfer
 - Low-level formatting creates **logical blocks** on physical media
- The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially
 - Sector 0 is the first sector of the first track on the outermost cylinder
 - Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost
 - Logical to physical address should be easy
 - ▶ Except for bad sectors
 - ▶ Non-constant # of sectors per track via constant angular velocity





HDD Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- Minimize seek time
- Seek time $\approx$ seek distance
- Disk **bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer





Disk Scheduling (Cont.)

- There are many sources of disk I/O request
 - OS
 - System processes
 - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
 - Optimization algorithms only make sense when a queue exists
- In the past, operating system responsible for queue management, disk drive head scheduling
 - Now, built into the storage devices, controllers
 - Just provide LBAs, handle sorting of requests
 - ▶ Some of the algorithms they use described next





Disk Scheduling (Cont.)

- Note that drive controllers have small buffers and can manage a queue of I/O requests (of varying “depth”)
- Several algorithms exist to schedule the servicing of disk I/O requests
- The analysis is true for one or many platters
- We illustrate scheduling algorithms with a request queue (0-199)

98, 183, 37, 122, 14, 124, 65, 67

Head pointer 53



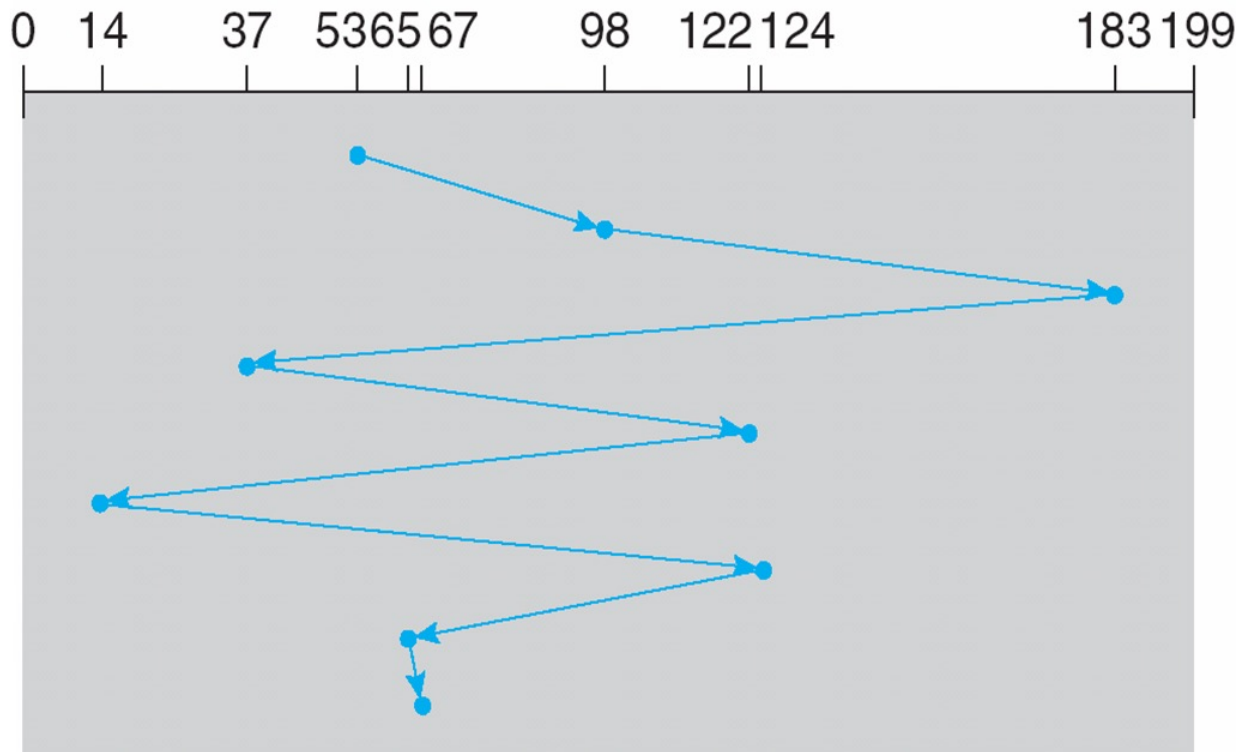


FCFS

Illustration shows total head movement of 640 cylinders

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53





SCAN

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- **SCAN algorithm** Sometimes called the **elevator algorithm**
- Illustration shows total head movement of 208 cylinders
- But note that if requests are uniformly dense, largest density at other end of disk and those wait the longest

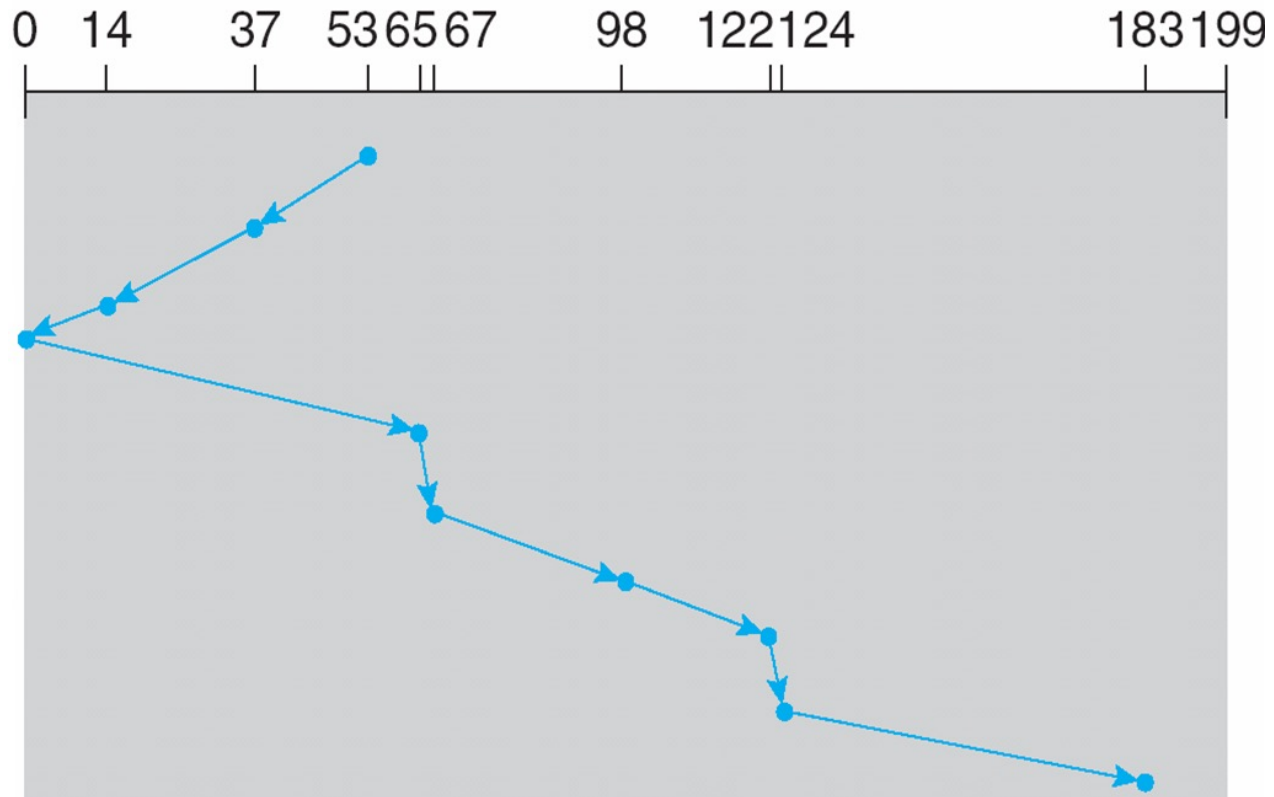




SCAN (Cont.)

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53





C-SCAN

- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes
 - When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one
- Total number of cylinders?

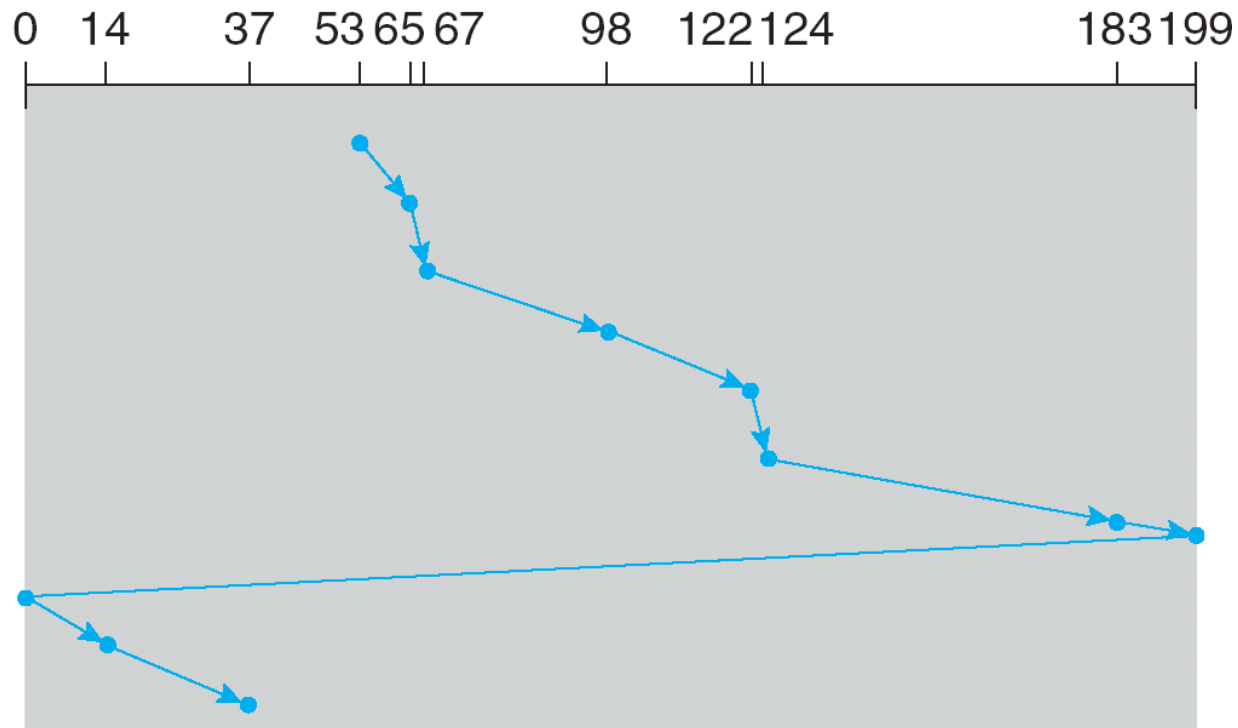




C-SCAN (Cont.)

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53





Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
 - Less starvation, but still possible
- To avoid starvation Linux implements **deadline** scheduler
 - Maintains separate read and write queues, gives read priority
 - ▶ Because processes more likely to block on read than write
 - Implements four queues: 2 x read and 2 x write
 - ▶ 1 read and 1 write queue sorted in LBA order, essentially implementing C-SCAN
 - ▶ 1 read and 1 write queue sorted in FCFS order
 - ▶ All I/O requests sent in batch sorted in that queue's order
 - ▶ After each batch, checks if any requests in FCFS older than configured age (default 500ms)
 - If so, LBA queue containing that request is selected for next batch of I/O
- In RHEL 7 also **NOOP** and **completely fair queueing** scheduler (**CFQ**) also available, defaults vary by storage device





RAID





RAID Structure

- **RAID – redundant array of inexpensive disks**
 - multiple disk drives provides reliability via **redundancy**
- Increases the **mean time to failure**
- **Mean time to repair** – exposure time when another failure could cause data loss
- **Mean time to data loss** based on above factors
- If mirrored disks fail independently, consider disk with 1300,000 **mean time to failure** and 10 hour mean time to repair
 - Mean time to data loss is $100,000^2 / (2 * 10) = 500 * 10^6$ hours, or 57,000 years!
- Frequently combined with **NVRAM** to improve write performance
- Several improvements in disk-use techniques involve the use of multiple disks working cooperatively





RAID (Cont.)

- Disk **striping** uses a group of disks as one storage unit
- RAID is arranged into six different levels
- RAID schemes improve performance and improve the reliability of the storage system by storing redundant data
 - **Mirroring** or **shadowing** (**RAID 1**) keeps duplicate of each disk
 - Striped mirrors (**RAID 1+0**) or mirrored stripes (**RAID 0+1**) provides high performance and high reliability
 - **Block interleaved parity** (**RAID 4, 5, 6**) uses much less redundancy
- RAID within a storage array can still fail if the array fails, so automatic **replication** of the data between arrays is common
- Frequently, a small number of **hot-spare** disks are left unallocated, automatically replacing a failed disk and having data rebuilt onto them





RAID Levels



(a) RAID 0: non-redundant striping.



(b) RAID 1: mirrored disks.



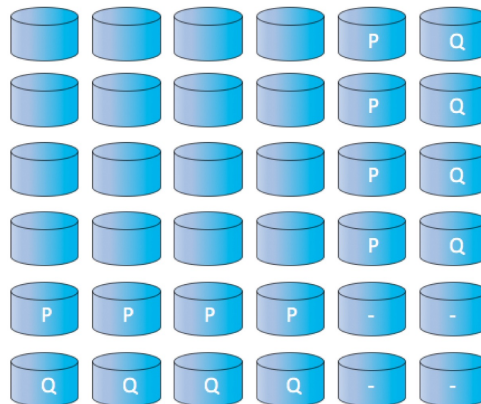
(c) RAID 4: block-interleaved parity.



(d) RAID 5: block-interleaved distributed parity.



(e) RAID 6: P + Q redundancy.



(f) Multidimensional RAID 6.





Raid 0: Striping

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Figure 38.1: **RAID-0: Simple Striping**

- สมมุติ 1 Block = 4KB





Raid 0: Striping

Disk 0	Disk 1	Disk 2	Disk 3	
0	2	4	6	chunk size: 2 blocks
1	3	5	7	
8	10	12	14	
9	11	13	15	

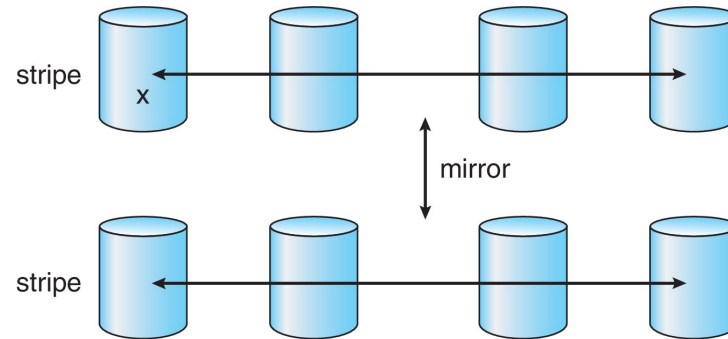
Figure 38.2: Striping With A Bigger Chunk Size

- สมมุติ 1 Block = 4KB
- 1 chunk = 2 blocks

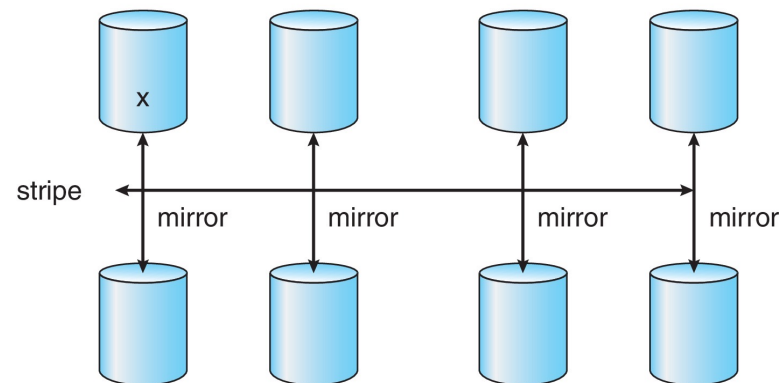




RAID (0 + 1) and (1 + 0)



a) RAID 0 + 1 with a single disk failure.



b) RAID 1 + 0 with a single disk failure.





Raid 10

Disk 0	Disk 1	Disk 2	Disk 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

Figure 38.3: Simple RAID-1: Mirroring

- สมมุติ 1 Block = 4KB





Raid 4

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

Figure 38.4: RAID-4 With Parity

- สมมุติ 1 Block = 4KB





Parity Calculation

C0	C1	C2	C3	P
0	0	1	1	$XOR(0,0,1,1) = 0$
0	1	0	0	$XOR(0,1,0,0) = 1$

- สมมุติ 1 Block = 1 bit
- ใช้ bitwise XOR
- ถ้าจำนวน 1 เป็นคี่ parity = 1
- ถ้าจำนวน 1 เป็นคู่ parity = 0
- จำนวน 1 รวม parity เป็นคู่เสมอ





Disk Reconstruction

C0	C1	C2	C3	P
0	0		1	$\text{XOR}(0,0,1,1) = 0$
0	1	0	0	$\text{XOR}(0,1,0,0) = 1$

- สมมุติ 1 Block = 1 bit
- ถ้า C2 เสีย ก็อ่านค่าทุกจากทุก Disks รวม Parity Disk มา XOR ก็จะกู้คืนได้





Parity Calculation

Block0	Block1	Block2	Block3	Parity
00	10	11	10	11

- สมมุติ 1 Block = 2 bits

Block0	Block1	Block2	Block3	Parity
00	10	11	10	11
10	01	00	01	10

- สมมุติ 1 Block = 4 bits





Raid 4 Block Update

C0	C1	C2	C3	P
0	0	1	1	$\text{XOR}(0,0,1,1) = 0$

$$P_{new} = (C_{old} \oplus C_{new}) \oplus P_{old}$$

- Additive Parity VS Subtractive Parity





Raid 4's Problem

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
*4	5	6	7	+P1
8	9	10	11	P2
12	*13	14	15	+P3

Figure 38.6: Example: Writes To 4, 13, And Respective Parity Blocks

- สมมุติ 1 Block = 4KB
- RAID 4 มี bottleneck ที่ Parity Disk (Access)





Raid 5

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

Figure 38.7: RAID-5 With Rotated Parity

- สมมุติ 1 Block = 4KB
- RAID 5 กระจาย Parity blocks



End of Lecture 13

